When Configuration Matters More Than Algorithm: An Empirical Evaluation of Lightweight Drift Detectors Under Resource Constraints

Stephen Kwaku Azongo[1], Livingston Seble[1], Alex Teye Ametepey[1], Kofi Sarpong Adu-Manu[1], Sandro Amofa[2]
[1]Department of Computer Science, University of Ghana
[2]Department of Computer Science, University of Cape Coast

***Abstract*—Concept drift (the change over time in the joint distribution $P(X, y)$ of data and labels) is a fundamental challenge for machine learning systems deployed in dynamic environments. While numerous drift detection algorithms have been proposed and benchmarked, existing evaluations hold resource conditions constant, obscuring a practical question of primary importance: when monitoring systems operate under constraints on monitoring frequency, detection window size, and reference window size, which constraints matter most? This paper presents a systematic empirical evaluation of three lightweight, distribution-based drift detectors, namely the Kolmogorov-Smirnov (KS) test, the Population Stability Index (PSI), and the Wasserstein distance, across 8,100 experimental runs spanning three real-world tabular datasets, three drift types (sudden, gradual, recurring), and a full factorial grid of resource constraint parameters. Non-parametric statistical testing (Kruskal-Wallis, Mann-Whitney U, Bonferroni correction) with effect size estimation (rank-biserial r, Spearman's ρ) establishes three principal findings: (1) constraint configuration produces a 37-fold range in detection delay, dwarfing the between-detector performance gap; (2) gradual drift is substantially harder to detect than sudden or recurring drift ($r \approx 0.30$, medium effect); and (3) Random Forest is significantly more robust to drift-induced accuracy degradation than LightGBM ($r = 0.383$) or Logistic Regression ($r = 0.185$), an advantage that holds across a range of hyperparameter settings but narrows under imbalance-aware metrics such as AUC. The findings yield a priority hierarchy for deployment decisions: monitoring regime configuration first, drift type awareness second, algorithm and model selection third.**

***Index Terms*—concept drift detection, resource-constrained machine learning, distributional shift, monitoring systems, empirical evaluation**

I. Introduction

Machine learning models deployed in production environments face a fundamental threat to their reliability: the world changes. Credit applicant populations shift as economic conditions evolve; network traffic changes as user behaviour and attack strategies adapt; equipment sensor signatures drift as components age [1]. When the joint distribution $P(X, y)$ that generated the training data no longer represents the incoming deployment data, a phenomenon formalised as *concept drift* [1], model predictions degrade, often silently and without obvious external signal.

The response to concept drift is monitoring: continuous statistical comparison of incoming data against a reference distribution, so that significant shifts trigger alerts and corrective action (retraining, recalibration, or human review). A mature literature on drift detection algorithms exists, encompassing error rate-based methods (DDM, EDDM, ADWIN), data distribution-based methods (KS test, PSI, Wasserstein), and multiple hypothesis test approaches [2]. Large-scale empirical comparisons have established the relative performance of these algorithms under controlled benchmark conditions [3], [4].

However, a consequential gap persists: existing benchmarks evaluate detectors under idealised conditions (unlimited reference memory, continuous monitoring, large detection windows) that do not reflect the resource constraints of many real-world deployment environments. Practitioners designing monitoring pipelines for edge devices, batch processing systems, or cloud-cost-constrained infrastructure must make explicit tradeoffs: how often to run detection checks, how much recent data to accumulate for each check, and how much historical reference data to retain. The quantitative consequences of these decisions on detection performance are not characterised by the existing literature.

This paper addresses that gap through a systematic, constraint-aware empirical evaluation. Our contributions are:

1) **A constraint-performance surface** for three lightweight distribution-based detectors across 8,100 experimental runs (1,620 distinct configurations $\times$ 5 repetitions): the first study to treat monitoring frequency, window size, and reference size as systematic experimental variables.
2) **A quantitative ranking** of constraint parameters by their effect on detection delay and missed detection rate, enabling evidence-based deployment design.
3) **A model robustness comparison** across three ML architectures (Random Forest, Logistic Regression, LightGBM) under controlled drift, establishing which model degrades most gracefully when distributional conditions change.

II. Related Work

Theoretical foundations. Gama et al. [1] provide the foundational formal treatment of concept drift, defining it as $\exists X : P_{t_0}(X, y) \neq P_{t_1}(X, y)$ and classifying drift patterns as sudden, gradual, incremental, and recurring. Lu et al. [2] extend this with a three-source taxonomy (virtual drift affecting $P(X)$, real drift affecting $P(y|X)$, and simultaneous change) and a four-stage detection framework: data retrieval $\rightarrow$ data

modelling → test statistic → hypothesis test. Both treatments emphasise that the parametrisation of data retrieval (window sizes) is as consequential as the choice of test statistic, a claim this study quantifies.

Empirical comparisons. Barros & Santos [3] compare 14 supervised drift detector configurations across six synthetic generators and seven dataset sizes, finding that RDDM and HDDM_A perform best overall but that no single detector dominates all conditions. Lukats et al. [4] benchmark 10 fully unsupervised detectors on 11 real-world streams, identifying D3, IBDD, and SPLL as leading approaches. Both studies hold resource conditions constant, leaving the constraint-performance interaction uncharacterised.

Distribution-based detection tools. The three detectors in this study, namely the KS test, PSI, and Wasserstein distance, have distinct theoretical properties. The KS test compares empirical CDFs non-parametrically [5]; PSI computes a symmetric KL-divergence on binned distributions with industry-calibrated thresholds [6]; Wasserstein distance measures the optimal transport cost between distributions [7]. Christen et al. [8] demonstrate their complementarity in an applied distributional comparison task. Their joint evaluation under constrained conditions has not been previously reported.

Resource constraints. Yang & Shami [9] identify monitoring frequency, memory, and labelling rate as primary constraint dimensions in deployed ML systems. Preuveneers et al. [10] examine resource-performance trade-offs for machine learning models in smart environments. Neither work provides a systematic empirical characterisation of constraint-performance tradeoffs for drift detection specifically.

III. Methodology

A. Datasets

Three publicly available tabular, binary-classification datasets are used (Table I). Datasets are chosen to represent diverse real-world domains with different class balance, feature types, and data volumes.

TABLE I
Dataset Summary

Dataset	Domain	Inst.	Feat.	Balance
Credit Default	Finance	30,000	23	78/22
Telco Churn	Telecom	7,043	20	73/27
Predictive Maint.	Industrial IoT	10,000	14	97/3

All categorical features are one-hot encoded; continuous features are standardised to zero mean and unit variance using parameters estimated on the reference split only, preventing leakage.

B. Drift Simulation

Each dataset is split into a reference period (first 60% of instances) and a stream period (remaining 40%). This ratio is not a convention established elsewhere in the drift detection literature; a review of window-based concept drift studies found that the field favours adaptive, variable-length windowing schemes (e.g., ADWIN) over a single fixed split ratio, so no canonical value exists to adopt. The 60/40 choice instead reflects a direct methodological tradeoff: a larger reference period gives a more stable estimate of the pre-drift distribution, while a larger stream period leaves more room to observe detection delay across the widest constraint configurations tested (window sizes up to 1,000 samples). This tradeoff was tested directly with a supplementary sensitivity analysis at 50/50 and 70/30 splits ($n = 405$ runs, 3 datasets $\times$ 3 detectors $\times$ 3 drift types $\times$ 5 repetitions per split, at a representative constraint setting of monitoring_frequency $= 100$, window_size $= 500$). The detector ranking established in Section IV-A is significant at every split tested (Kruskal-Wallis $p < 0.0001$ at 50/50, 60/40, and 70/30 alike), and detection delay for KS and PSI does not differ significantly across split ratios ($p = 0.60$ and $p = 0.78$ respectively). Wasserstein shows a small but statistically detectable sensitivity to split ratio ($p = 0.004$; mean delay 180.7 at 50/50, 153.6 at 60/40, 181.7 at 70/30), which does not change any comparative conclusion in this paper but is noted here for completeness. Drift is injected into a single designated feature per dataset (AGE for Credit Default, Monthly Charges for Telco Churn, Tool wear for Predictive Maintenance), chosen as the feature most likely to exhibit natural distributional shift in a deployed system. This univariate injection design isolates each detector's sensitivity to a single-feature shift from the added complexity of multivariate drift. In all three drift types, injection begins at 50% of the stream length, and the shift magnitude is expressed in units of the feature's own standard deviation σ to keep magnitudes comparable across datasets with different natural scales. A magnitude of 2σ was chosen following the variance-based synthetic perturbation methodology of Rabanser et al. [11], whose "Failing Loudly" benchmark established this class of standard-deviation-scaled shift as a standard tool for constructing detectable, controllable synthetic drift in dataset-shift detection studies:

- **Sudden drift:** An instantaneous, permanent additive shift applied from the injection point onward: $x_i' = x_i + 2\sigma$ for all i at or after the injection point.
- **Gradual drift:** The shift ramps linearly from 0 to 2σ over a fixed transition window of 500 samples following the injection point, then remains constant at 2σ for the rest of the stream.
- **Recurring drift:** The shift alternates on and off in fixed cycles of 200 samples each (2σ for 200 samples, 0 for the next 200, repeating), simulating seasonal or periodic concept re-emergence.

C. Drift Detectors

Three lightweight, label-free, distribution-based detectors are implemented, each operating on the single drift-target feature per experimental run:

KS Test: The two-sample Kolmogorov-Smirnov statistic $D = \max_x |F_{ref}(x) - F_{curr}(x)|$ is computed. Drift is declared if the p-value falls below $\alpha = 0.05$.

PSI: The feature is discretised into 10 equal-frequency bins computed on the reference data, with the outermost bin edges extended to $\pm\infty$ so that out-of-range monitoring-period values remain binnable. $PSI = \sum_i (O_i - E_i) \times \ln(O_i/E_i)$ is computed; drift is declared if PSI exceeds 0.25 (the standard threshold for major shift).

Wasserstein Distance: The first-order Wasserstein distance $W_1(P_{ref}, P_{curr})$ is computed as the area between sorted empirical CDFs. Drift is declared if the distance exceeds a fixed threshold of 0.25, set once via preliminary analysis of the pre-drift noise floor on standardised data (pre-drift 99th percentile ≈ 0.24) so as to keep the false-positive rate below 1% while remaining sensitive to a 2σ shift.

D. Constraint Grid

Three resource constraint parameters are varied in a full factorial design:

- **Monitoring frequency (F):** $\{10, 50, 100, 500, 1000\}$ samples between successive detection checks
- **Detection window size (W):** $\{50, 100, 500, 1000\}$ samples used in each check
- **Reference window size (R):** $\{100, 500,$ unlimited (full reference period)$\}$

Crossed with 3 datasets, 3 detectors, and 3 drift types, this yields $3 \times 3 \times 3 \times 5 \times 4 \times 3 = 1{,}620$ distinct experimental configurations. Each configuration is replicated 5 times with independent random seeds, giving $1{,}620 \times 5 = 8{,}100$ total experimental runs.

These ranges are grounded in the operational vocabulary of production ML monitoring rather than an arbitrary log-scale sweep. Industry monitoring guidance (e.g., Azure's Machine Learning and Evidently AI's production monitoring documentation) does not prescribe a single correct check interval; it instead frames the choice as a tradeoff between near-continuous and batch-style monitoring, driven by data volume and compute budget. The five monitoring-frequency levels tested span exactly this range: $F = 10$ approximates near-continuous, streaming-style monitoring; $F = 1000$ approximates the coarse, periodic batch monitoring (e.g., daily or weekly jobs on accumulated logs) common in cost-constrained deployments; the intermediate levels (50, 100, 500) interpolate between these operating points. Window size and reference size are varied over a comparable range for the same reason, so that the grid characterises the full space from best-case to worst-case resourcing rather than a single assumed operating point.

E. Evaluation Metrics

- **Missed Detection Rate (MDR):** proportion of drift events for which no detection occurs within the monitoring period.
- **Detection Delay:** mean number of monitoring windows elapsed between drift injection and the first detection event, computed across repetitions.
- **Computational Cost:** total CPU time (ms) consumed by the detection procedure over the full monitoring period.

F. Statistical Analysis

All pairwise and multi-group comparisons use non-parametric tests: the Kruskal-Wallis H-test for multi-group comparisons, Mann-Whitney U with Bonferroni correction for post-hoc pairwise tests. Effect sizes are reported as rank-biserial correlation r (for pairwise tests) and Spearman's ρ (for constraint-performance correlations). Significance threshold: $\alpha = 0.05$.

Statistical power. Effective sample sizes (achieved observations per group) vary substantially by comparison family: detector and drift-type comparisons pool 2,645–2,700 observations per group; constraint correlations use all 8,045 detected runs; model-comparison pairs use 135 observations per group. Using Noether's (1987) large-sample formula for the Wilcoxon–Mann–Whitney test, the minimum detectable rank-biserial effect at 80% power and $\alpha = 0.05$ is $r \approx 0.031$ for the detector and drift-type families and the constraint correlations, but $r \approx 0.139$ for the smaller model-comparison family. This bears directly on how the non-significant results should be read: the null finding for sudden-versus-recurring drift (rank-biserial effect size $r = 0.012$) and for reference size (Spearman correlation $\rho = 0.012$) both fall well below their respective detection thresholds, supporting genuine equivalence rather than an underpowered non-result. The KS-versus-Wasserstein null ($r = -0.029$), however, sits close to its detection threshold and should be read as "no detectable difference at this sample size" rather than definitive proof of equivalence. The same applies to the LightGBM-versus-Logistic-Regression comparison under the absolute degradation metric ($r = -0.136$, against a threshold of 0.139), a result revisited in Section IV-D.

Correction across the full test family. The Bonferroni correction described above is applied within each family of pairwise comparisons (e.g., the three detector pairs). As a further check on the family-wise error rate across the study as a whole, Benjamini–Hochberg false discovery rate correction was applied jointly to all twelve headline comparisons reported in this paper (three detector pairs, three drift-type pairs, three model pairs, three constraint correlations). Every comparison significant under within-family Bonferroni correction remained significant under this stricter, study-wide correction, and every non-significant comparison remained non-significant. None of the paper's conclusions depend on the choice between these two correction schemes.

G. Model Comparison

Three classification models are trained on the reference period and evaluated for performance degradation in the post-drift monitoring period: Random Forest (100 trees, unlimited depth, scikit-learn defaults), Logistic Regression (L2 regularisation, $C = 1.0$, scikit-learn defaults), and LightGBM (100 estimators, learning rate 0.1, LightGBM defaults). No hyperparameter tuning was performed; all three models use their library default settings aside from the number of estimators, so that the comparison reflects each algorithm's baseline inductive bias rather than a dataset-tuned configuration. Degradation

is measured primarily as the relative accuracy change, $\Delta_{rel} = (Acc_{ref} - Acc_{post})/Acc_{ref}$, which normalises for baseline accuracy differences across models so that a model with lower reference-period accuracy is not automatically credited with "robustness" simply for having less room to degrade. Absolute accuracy difference, $\Delta_{abs} = Acc_{ref} - Acc_{post}$, is reported as a secondary metric to capture the practical impact on prediction quality.

No internal validation split was used when fitting the reference-period models, since hyperparameters were fixed in advance rather than tuned. As a validity check, a supplementary 80/20 split of the reference period was used to test for overfitting: Random Forest reaches 99.9–100% in-sample accuracy but only 79–98% held-out accuracy across the three datasets, a substantially larger train-validation gap than LightGBM or Logistic Regression show. This raises a natural question, whether Random Forest's drift-robustness advantage depends on this specific, unregularised configuration, which is addressed directly in Section IV-D via a hyperparameter sensitivity check. Two further supplementary analyses are also reported there: a check of whether the model ranking is sensitive to the exact hyperparameter settings used, and a check of whether it is sensitive to the choice of degradation metric, using balanced accuracy and ROC-AUC in addition to raw accuracy to account for the class imbalance present in all three datasets. Both supplementary analyses use a smaller replication of the model-comparison design ($n = 45$ observations per model: 3 datasets $\times$ 3 drift types $\times$ 5 repetitions), since detector choice does not affect model accuracy and is therefore held constant rather than repeated.

H. Implementation Details

All experiments were implemented in Python 3.13.5, using scikit-learn 1.8.0 for the Random Forest and Logistic Regression classifiers and for train/test splitting, LightGBM 4.6.0 for the gradient boosting classifier, scipy 1.17.1 for the statistical tests (Kolmogorov-Smirnov, Kruskal-Wallis, Mann-Whitney U, Spearman correlation), pandas 3.0.2 and numpy 2.4.4 for data handling, and matplotlib 3.10.8 with seaborn 0.13.2 for the figures. Model fitting used scikit-learn's and LightGBM's built-in multi-threading (`n_jobs=-1`); the detection procedure itself, and the sequencing of configurations across the experimental grid, ran single-threaded and sequentially, with no distributed or multi-process execution.

Experiments were run on a single consumer laptop: an Intel Core i3-1125G4 (4 physical cores, 8 logical threads, 2.00 GHz base clock) with 4 GB RAM, running Windows 11 Home. This is deliberately modest hardware relative to typical research computing environments, consistent with the paper's resource-constrained framing. Total wall-clock time was not logged during the original run; a benchmark of a random 50-configuration sample from the full grid on the same machine gives approximately 0.98 seconds per run, extrapolating to approximately 2.2 hours for the complete 8,100-run grid. The reported computation-time metric (Table II) measures only the detection procedure itself (CPU time consumed by the statistical test), and is therefore a property of the detector algorithms rather than of this specific hardware; the 2.2-hour figure includes model training and drift injection overhead in addition to detection, and is reported here only to establish the practical feasibility of the full study on ordinary hardware.

IV. Results

A. Detector Performance (RQ1)

Table II summarises mean performance across all datasets, drift types, and constraint configurations. Wasserstein Distance achieves the lowest missed detection rate and detection delay; PSI achieves the lowest computational cost. The omnibus Kruskal-Wallis test across all three detectors is significant ($H = 148.739$, $p < 0.001$), but this is driven entirely by PSI: PSI differs significantly from both KS ($r = 0.151$, $p < 0.001$) and Wasserstein ($r = -0.177$, $p < 0.001$), while KS and Wasserstein are statistically indistinguishable from each other ($p = 0.186$, Bonferroni-corrected, $r = -0.029$). Effect sizes across all significant pairs are small ($|r| \leq 0.18$), indicating modest absolute differences. The KS-Wasserstein null result sits close to the study's detection threshold for this sample size (minimum detectable $r \approx 0.031$ at 80% power; Section III-F), so it is best read as "no detectable difference," not proof that the two are exactly equivalent.

TABLE II
Detector Performance (mean across all conditions)

Detector	MDR	Delay (windows)	Compute (ms)
KS Test	1.11%	216.28	378.51
PSI	0.56%	291.73	77.53
Wasserstein	0.37%	205.52	114.44

Critically, the between-detector performance gap is substantially smaller than the within-detector variation induced by constraint configuration (Section IV-C), establishing that algorithm selection is a secondary consideration relative to system configuration.

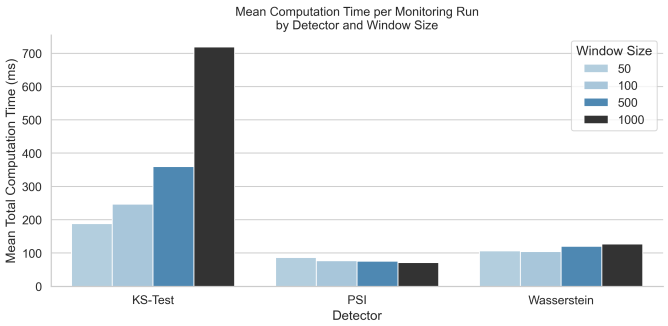


Fig. 1. Mean total computation time (ms) per monitoring run, by detector and window size. PSI's cost is effectively independent of window size (fixed bin structure); KS scales sharply with window size due to repeated sorting and hypothesis-test overhead.

B. Drift Type Effects (RQ2)

Drift type is the strongest single predictor of detection performance in the study (Kruskal-Wallis $H = 504.546$, $p < 0.001$, $r \approx 0.30$, medium effect). Fig. 2 shows the detection delay distribution by drift type and detector.

Sudden and recurring drift are statistically indistinguishable from each other across all detectors and constraint configurations (Mann-Whitney $p = 1.00$, $r \approx 0.00$). Gradual drift produces significantly longer detection delays than both sudden drift ($p < 0.001$, $r = 0.302$) and recurring drift ($p < 0.001$, $r = 0.308$). All post-hoc tests are Bonferroni-corrected.

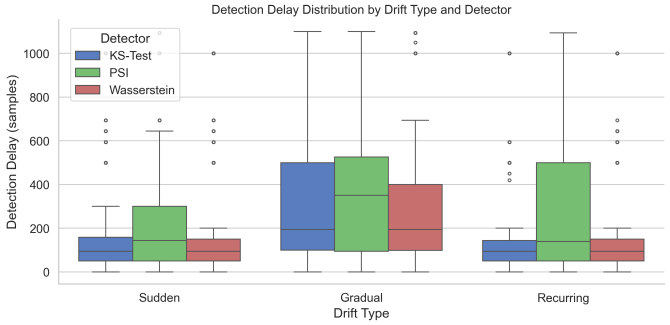


Fig. 2. Detection delay distribution (boxplot) by drift type and detector. Gradual drift shows both a higher median delay and a wider spread than sudden or recurring drift, across all three detectors.

The equivalence of sudden and recurring drift detection performance is a notable result: distribution-based detectors respond to the return of an old concept with the same sensitivity as to an entirely new sudden change. This occurs because both patterns produce an abrupt shift in the current window's distribution relative to the reference window, regardless of whether the new distribution has been seen before.

Gradual drift's resistance to detection is attributable to the dilution of the distributional signal during the transition period: when old and new distributions co-occur, the test statistics computed on the mixed detection window are intermediate in value, often falling below the detection threshold throughout the transition. All three detectors are affected equally, confirming that gradual drift resistance is a property of the detection paradigm rather than of any particular test statistic.

C. Resource Constraint Effects (RQ3)

Table III reports the Spearman correlation and Kruskal-Wallis H-statistic between each constraint parameter and detection delay, providing a quantitative ranking of constraint importance.

Detection window size has the strongest individual effect on detection delay: larger windows require more samples to accumulate before each check, directly lengthening the minimum time from drift onset to first detection. Monitoring frequency has the second strongest effect: lower frequency means fewer detection opportunities per unit time, extending the expected time between drift onset and the next scheduled check.

TABLE III
CONSTRAINT PARAMETER EFFECTS ON DETECTION DELAY

Constraint	ρ	H	p
Window size	0.543	2,381.184	< 0.001
Monitoring frequency	0.487	1,948.767	< 0.001
Reference size	0.012	50.015	< 0.001

Reference window size shows a statistically significant but practically small correlation ($\rho = 0.012$). Closer inspection reveals a threshold effect rather than a monotone relationship: the 100-sample reference window produces substantially degraded performance (insufficient samples to estimate the reference distribution reliably), while the 500-sample reference window is functionally equivalent to the unlimited reference window. Practitioners can achieve full reference-quality detection with as few as 500 stored reference samples.

The combined effect of constraint configuration produces a 37-fold range in detection delay: from a mean of 22.0 monitoring windows under the most favourable configuration (F=10, W=50, R=unlimited) to 827.6 windows under the least favourable (F=1000, W=1000, R=100). This 37-fold range substantially exceeds the between-detector performance variation and establishes constraint configuration as the primary determinant of monitoring effectiveness.

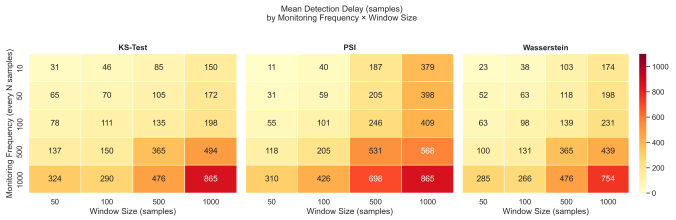


Fig. 3. Mean detection delay (samples) by monitoring frequency × window size, faceted by detector. The pattern is consistent across all three detectors: delay grows with both coarser monitoring frequency and larger window size, and the two effects compound.

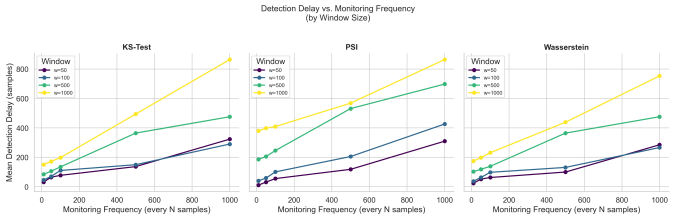


Fig. 4. Detection delay as a function of monitoring frequency, with one line per window size, faceted by detector. The near-parallel, non-overlapping curves show that window size and monitoring frequency act as largely independent, additive levers on detection delay.

D. Model Robustness Under Drift (RQ4)

Across the 405 model comparison experiments, Random Forest is significantly more robust to drift-induced accuracy degradation than either LightGBM or Logistic Regression

under the relative-degradation metric (Table IV). The RF vs. LightGBM effect ($r = 0.383$) is medium by conventional benchmarks; the RF vs. LR effect ($r = 0.185$) is small but statistically reliable. Both effect sizes are numerically identical to those obtained under the absolute-degradation metric, indicating that Random Forest's advantage is not an artefact of baseline accuracy differences between models. Unlike the absolute metric, however, the relative metric also finds LightGBM significantly more robust than Logistic Regression ($r = -0.185$, $p = 0.025$, Bonferroni-corrected), a difference the absolute metric could not distinguish from chance ($p = 0.160$). Correcting for the baseline-accuracy confound therefore sharpens rather than changes the finding: it yields a complete, significant three-way ordering, Random Forest > LightGBM > Logistic Regression. This is consistent with the power analysis in Section III-F: at $n = 135$ per group, the minimum detectable effect at 80% power is $r \approx 0.139$, and the absolute-metric LightGBM-vs-LR effect ($r = -0.136$) sat almost exactly on that boundary. This explains why it read as non-significant under the absolute metric despite the underlying difference being real enough to cross the threshold ($r = -0.185$) once the baseline-accuracy confound was removed.

TABLE IV
MODEL ROBUSTNESS TO DRIFT-INDUCED DEGRADATION (RELATIVE-DEGRADATION METRIC)

Comparison	p (Bonf.)	r	Effect
RF vs. LightGBM	< 0.001	-0.383	Medium
RF vs. LR	0.025	-0.185	Small
LightGBM vs. LR	0.025	-0.185	Small

The model comparison results are consistent with the hypothesis that gradient boosting's sequential, residual-fitting structure, which enables high accuracy in stationary conditions, also creates greater sensitivity to distributional shift, because each tree corrects the residual of the ensemble built so far rather than averaging independently over the training distribution. Random Forest's bagging and per-split feature randomisation decorrelate its constituent trees, so that a shift affecting one feature's split is not shared by the whole ensemble.

Hyperparameter sensitivity. Random Forest's held-out validation diagnostic (Section III-G) showed substantial overfitting at its default unlimited tree depth, raising the question of whether its robustness advantage depends on that specific configuration. This was tested directly by rerunning the drift-injection procedure with Random Forest at max_depth $\in \{3, 5, 10, 20, \text{unlimited}\}$ and comparing each against LightGBM's established degradation ($n = 45$ per group). Random Forest's advantage holds at every depth tested (all $p < 0.05$, Bonferroni-corrected), and is, if anything, stronger under regularisation: mean accuracy degradation is 0.0033 at depth 3 and 0.0044 at depth 5, both markedly below the 0.0150 observed at unlimited depth, and well below LightGBM's 0.0218. A parallel check varying LightGBM's learning rate (0.05, 0.1, 0.2) found Random Forest (unlimited depth) significantly more robust than LightGBM at every rate tested (all $p < 0.005$). The robustness advantage is therefore not an artefact of the specific hyperparameters used in the main experiment; it holds across the tested hyperparameter space for both models, and strengthens under regularisation rather than depending on the absence of it.

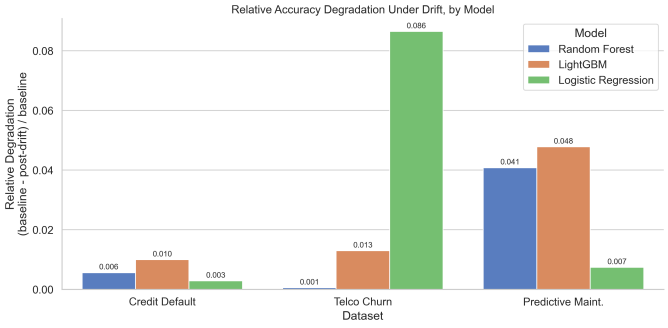


Fig. 5. Mean relative accuracy degradation ((baseline − post-drift) / baseline) by model, per dataset. Random Forest's advantage over LightGBM holds in direction across all three datasets. Logistic Regression's poor aggregate performance is driven almost entirely by Telco Churn, where it is the least robust model by a wide margin; on the other two datasets it is competitive with or better than both ensembles.

Robustness under imbalance-aware metrics. All three datasets are class-imbalanced, and raw accuracy is dominated by majority-class performance. To test whether the accuracy-based ranking generalises, degradation was additionally measured using balanced accuracy (the mean of sensitivity and specificity) and ROC-AUC, computed on the same $n = 45$ replication (Table V). The picture is more nuanced than the accuracy-based result alone suggests. Balanced accuracy improves post-drift for all three models (Random Forest -0.036, LightGBM -0.013, Logistic Regression -0.057; negative values indicate improvement), suggesting the injected drift shifts predictions toward better minority-class recall at some cost to majority-class accuracy, a trade-off raw accuracy does not capture. Under ROC-AUC, LightGBM again shows the largest degradation ($+0.028$), Random Forest an intermediate, significantly smaller degradation ($+0.017$, $p < 0.001$ vs. LightGBM), and Logistic Regression the smallest change (-0.006), which is not statistically distinguishable from Random Forest at this sample size ($p = 0.26$).

TABLE V
MODEL DEGRADATION ACROSS METRICS ($n = 45$ PER MODEL)

Metric	RF	LightGBM	LR
Accuracy degradation	0.0150	0.0218	0.0265
Balanced accuracy degradation	−0.0360	−0.0130	−0.0566
AUC degradation	0.0171	0.0282	−0.0055

Across all three metrics, LightGBM is consistently the least robust model. The comparison between Random Forest and Logistic Regression, however, is metric-dependent: Random Forest's advantage over Logistic Regression is clear and sig-

nificant for raw and relative accuracy, but the two become statistically indistinguishable under AUC and balanced accuracy. This is consistent with the dataset-level pattern already established: Logistic Regression's poor aggregate accuracy is driven almost entirely by a single severe failure on Telco Churn, and that failure appears to weigh less heavily on rank-based and imbalance-corrected metrics than on raw accuracy. The choice of degradation metric should therefore match the deployment priority: where overall predictive accuracy is what matters operationally, Random Forest is the more robust choice; where minority-class detection is the priority, common in the exact domains these datasets represent (credit default, churn, machine failure), Logistic Regression's robustness is competitive with Random Forest's despite its markedly worse showing on raw accuracy.

V. Discussion

A. The Priority Hierarchy

Integrating the findings across all four research questions yields a priority hierarchy for resource-constrained drift monitoring deployment:

First priority: monitoring regime configuration. The constraint analysis demonstrates that monitoring frequency and detection window size collectively produce the largest performance variation in the study, a 37-fold range in detection delay, with effect sizes ($\rho \approx 0.49$–0.54) substantially larger than those for detector choice ($r \approx 0.15$). System designers should set monitoring frequency as high as their compute budget allows and choose the smallest detection window that provides acceptable statistical power for their expected drift magnitude.

Second priority: drift type awareness. Gradual drift is systematically and substantially harder to detect than sudden or recurring drift ($r \approx 0.30$, medium effect). A system calibrated and validated exclusively on sudden-change scenarios, as most benchmark studies are, may provide misleading performance guarantees for gradual drift, which is arguably more common in real deployment environments where populations shift slowly. Practitioners should explicitly test their monitoring configurations against gradual drift scenarios and consider lower detection thresholds or smaller window sizes if gradual drift is expected.

Third priority: algorithm and model selection. After the monitoring regime and drift type considerations are addressed, detector choice and model choice represent relatively smaller sources of performance variation. Wasserstein distance offers the best detection capability at moderate computational cost; KS offers strong detection at higher compute; PSI offers acceptable detection at lowest compute. Random Forest is the recommended base model in non-stationary environments where retraining is costly and overall accuracy is the operational priority; where minority-class detection matters most, its advantage over Logistic Regression narrows substantially (Section IV-D).

B. The Reference Window Threshold

The threshold finding for reference window size, that 500 samples is functionally equivalent to an unlimited reference window, has a direct practical implication for memory-constrained deployment. A monitoring system that stores and maintains 500 reference samples per feature achieves full detection quality without requiring access to the entire historical training dataset. This supports the design of streaming monitoring pipelines in which reference data can be pruned periodically without sacrificing detection performance.

C. Positioning Relative to Prior Work

This study's findings complement and extend the large-scale benchmark of Barros & Santos [3] in two directions. First, while Barros & Santos [3] find that no single detector dominates across all conditions, this study reveals that the between-detector differences are secondary to the effect of monitoring configuration, a finding that reframes the focus from algorithm selection to system design. Second, the gradual drift finding of this study, obtained for distribution-based label-free detectors, replicates the qualitative finding of Barros & Santos [3] for supervised error rate-based detectors, suggesting that gradual drift resistance is a detection-paradigm-level challenge rather than an algorithm-specific limitation.

The model robustness finding, that Random Forest outperforms LightGBM under drift, appears to contrast with the finding of Pathi [5] that gradient boosting achieves the best accuracy-stability balance in some datasets. The discrepancy is attributable to the difference in evaluation task: Pathi [5] measures PSI as a static population stability index, while this study measures dynamic accuracy degradation under experimentally injected drift. The two metrics capture different aspects of model stability, and both contribute to a more complete picture.

D. Limitations

This study focuses exclusively on tabular binary-classification tasks; the findings may not generalise to image, text, or multi-class problems. Drift is simulated via controlled statistical transformations rather than observed as a naturally occurring process; real-world drift may involve correlated multi-feature shifts that are qualitatively different. All three detectors are applied feature-wise (univariate), potentially missing multivariate distributional changes involving feature correlation structure. Monitoring configurations are evaluated across three constraint levels per parameter; finer resolution within the parameter space may reveal a more detailed tradeoff structure.

VI. Conclusion

This paper presents the first systematic constraint-aware evaluation of three lightweight distribution-based drift detectors across 8,100 experimental configurations. The central finding is that the configuration of the monitoring system, specifically monitoring frequency and detection window size, produces a 37-fold range in detection delay, substantially

exceeding the between-detector performance gap. This result establishes that the choice of *how* a monitoring system is configured is more consequential than the choice of *which* algorithm it uses.

Gradual drift is systematically and substantially harder to detect than sudden or recurring drift across all detectors and constraint configurations, with a medium effect size ($r \approx 0.30$) that significantly exceeds the detector-comparison effects. The equivalence of sudden and recurring drift detection performance indicates that distribution-based detectors respond to concept recurrence with the same sensitivity as to novel sudden shifts.

A reference window threshold of approximately 500 samples provides detection performance equivalent to an unlimited reference, supporting memory-efficient monitoring pipeline design. Random Forest shows the greatest robustness to drift-induced accuracy degradation among the three evaluated model architectures, an advantage confirmed across a range of hyperparameter settings, though it narrows under imbalance-aware metrics where Logistic Regression becomes competitive.

The practical guidance distilled from these findings is a ranked priority hierarchy: configure the monitoring regime first, account for drift type second, and select the detector and model third. Deployment decisions should be informed by constraint-aware evaluation rather than benchmark results obtained under idealised conditions that do not represent the resource environments of real production systems.

REFERENCES

[1] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2013.

[2] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang, "Learning under concept drift: A review," *IEEE Transactions on Knowledge and Data Engineering*, vol. 31, no. 12, pp. 2346–2363, 2019.

[3] R. S. M. Barros and S. G. T. C. Santos, "A large-scale comparison of concept drift detectors," *Information Sciences*, vol. 451–452, pp. 348–370, 2018.

[4] D. Lukats, O. Zielinski, A. Hahn, and F. Stahl, "A benchmark and survey of fully unsupervised concept drift detectors on real-world data streams," *International Journal of Data Science and Analytics*, vol. 19, pp. 1–31, 2024.

[5] N. Pathi, "Model evaluation beyond AUC: A comparative study of somers' D, KS statistic, and PSI," in *Proceedings of ICRTCSIT 2025*, 2025.

[6] R. Taplin and C. Hunt, "The population accuracy index: A new measure of population stability for model monitoring," *Risks*, vol. 7, no. 2, p. 53, 2019.

[7] B. R. Prashanth, A. Kumar, P. Puneetha, and S. Vinay, "Drift detection using bayesian neural networks and Wasserstein distance," in *IEEE International Conference on Advances in Machine Intelligence and Applications (AMATHE)*, 2025.

[8] P. Christen, W. Sabra, and E. Rahm, "Stop relearning: Model reuse via feature distribution analysis," in *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM)*, 2024.

[9] L. Yang and A. Shami, "A lightweight concept drift detection and adaptation framework for IoT data streams," *IEEE Internet of Things Magazine*, vol. 4, no. 2, pp. 96–101, 2021.

[10] D. Preuveneers, I. Tsingenopoulos, and W. Joosen, "Resource usage and performance trade-offs for machine learning models in smart environments," *Sensors*, vol. 20, no. 4, p. 1176, 2020.

[11] S. Rabanser, S. Günnemann, and Z. Lipton, "Failing loudly: An empirical study of methods for detecting dataset shift," *Advances in Neural Information Processing Systems*, vol. 32, pp. 1394–1406, 2019.